BSCS 3112 - ARTIFICIAL INTELLIGENCE – LAB

LESSON 7 – FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

OCTOBER 17, 2026

PRESENTED BY: PROF. ROB MALITAO

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Learning Objectives:

Knowledge (K) – Cognitive:

- Synthesize speech processing, agentic tool calling, relational database transactions, and Finite State Machines (FSM) into an integrated multi-modal AI agent.
- Analyze edge-case dialogue failures (e.g., out-of-stock items, speech recognition errors, state-skipping attempts) to engineer programmatic guardrails.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Learning Objectives:

Skills (S) – Psychomotor/Practical:

- Construct a voice-enabled I/O interface combining Speech-to-Text (STT) and Text-to-Speech (TTS) with an underlying stateful Python agent.
- Implement dynamic agentic tool calling where Qwen 2.5 autonomously queries and mutates SQLite database tables (menu_items, table_sessions, cart_items).
- Build a real-time ordering UI (Web Dashboard or Python GUI) that stays synchronized with database state changes and displays an active order tab.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Learning Objectives:

Attitude (A) – Affective:

- Demonstrate professional engineering collaboration through driver-navigator tandem programming.
- Value user-centric design principles, ensuring the AI agent remains courteous, transparent, and resilient during customer interactions.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Final Project Specification: AI Virtual Waiter for Perpetual Bistro Using Ollama and Qwen

- Execution Window: October 19 – November 6, 2026
- Final Project Due Date: November 6, 2026
- Live Demo Presentations: November 7, 2026
- Core Tech Stack: Python 3.10+, Local Ollama (Qwen 2.5), SQLite, STT/TTS Speech Modules, Flask/CustomTkinter

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

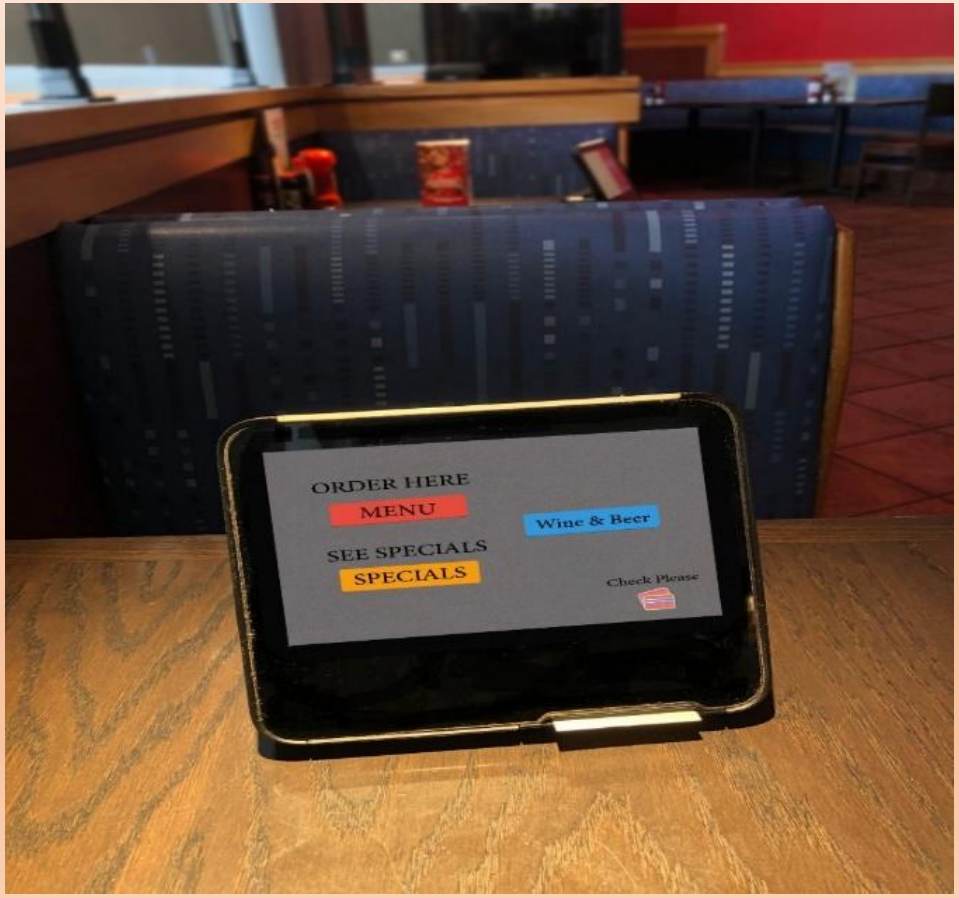
Project Overview & Business Scenario

Perpetual Bistro, the campus dining facility at UPHSD, seeks to modernize its customer service experience by deploying a table-side digital waiter. The system must run 100% locally (zero cloud dependencies) using Ollama (Qwen 2.5), Python, SQLite, and STT/TTS Speech Modules.

The AI Virtual Waiter serves as an interactive, voice-enabled, state-aware ordering assistant. Customers converse naturally—asking about menu ingredients, requesting customizations (e.g. *"Make it spicy, no onions"*), checking dietary restrictions, and managing their active order tab. The system must dynamically bind spoken or typed user intents to local SQLite tool functions, update real-time kitchen inventory, enforce strict Finite State Machine (FSM) state transitions, and generate a final itemized receipt upon checkout.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Sample images of the Application Platform:





Key Comparison Matrix: Lesson 6 vs Final Project

Feature / Dimension	Lesson 6 Lab Output	Final Project Output (Perpetual Bistro Capstone)
Primary Objective	Prove backend multi-turn FSM logic, state tracking, and basic SQLite tool execution.	Deploy an end-to-end, voice-enabled, table-side digital ordering assistant.
Input / Output Modality	Single-Modal (Text): Console CLI prompts (input()) or optional basic HTML text log.	Multi-Modal (Voice + Visual): Microphone Speech-to-Text (STT) + Text-to-Speech (TTS) voice responses + Graphical Dashboard.
User Interface (UI)	Command Line Interface (CLI) terminal output or basic single-page text form.	Full Web Dashboard (Flask) or Desktop GUI (CustomTkinter) with real-time cart table, visual menu grid, and item images.
Data Source	Hardcoded generic mock menu dataset seeded by the instructor.	Authentic Canteen Dataset: Real UPHSD canteen best-sellers, prices, photos, and allergen profiles gathered via student pair survey.
System Guardrails	Basic inventory checks (e.g., stopping orders when stock_qty = 0).	Comprehensive Guardrails: Handles out-of-stock items, post-confirmation edit blocks, out-of-domain query redirection, and panel stress tests.
Deliverables Scope	3 core Python/SQL scripts (fsm_agent.py, db_tools.py, init_restaurant_db.sql) + test log.	Full multi-file repository (app.py, speech_engine.py, ui_dashboard/, video walkthrough, architecture markdown, live defense).

Key Comparison Matrix: Lesson 6 vs Final Project

- "In **Lesson 6 Lab**, you build the waiter's brain and memory using text commands. In the **Final Project**, you give the waiter a voice (STT/TTS), eyes (UI Dashboard with canteen photos), and put them to work at a real table in Perpetual Bistro!"

Project Milestones & Execution Schedule:

Milestone	Deliverables	Due Date	Weight
Milestone 1	Voice I/O Pipeline, FSM Core Controller, and SQLite Tool Engine (restaurant_db.sql, db_tools.py, fsm_controller.py, speech_engine.py)	Wednesday, Oct 28, 2026	40%
Milestone 2	Integrated UI Dashboard, Guardrails, Final Codebase, Documentation, and Walkthrough Video (app.py, ui_dashboard/, documentation.md)	Saturday, Nov 06, 2026	60%
Live Defense	10-Minute Live Demonstration and Faculty Panel Defense	Saturday, Nov 7, 2026 (Class Hours)	Final Exam

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Required Tech Stack & Setup Rules

System Requirements & Environment:

1. Local LLM Engine: Ollama running qwen2.5 (qwen2.5:7b recommended; qwen2.5:3b for 8GB RAM laptops).
2. Core Language & Database: Python 3.10+ and SQLite 3 (restaurant.db).
3. Voice/Speech Libraries: Speech-to-Text (e.g., whisper / speech_recognition) and Text-to-Speech (e.g., pyttsx3).
4. UI Dashboard Framework: Web GUI (Flask + HTML/Tailwind CSS) OR Desktop GUI (CustomTkinter).

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

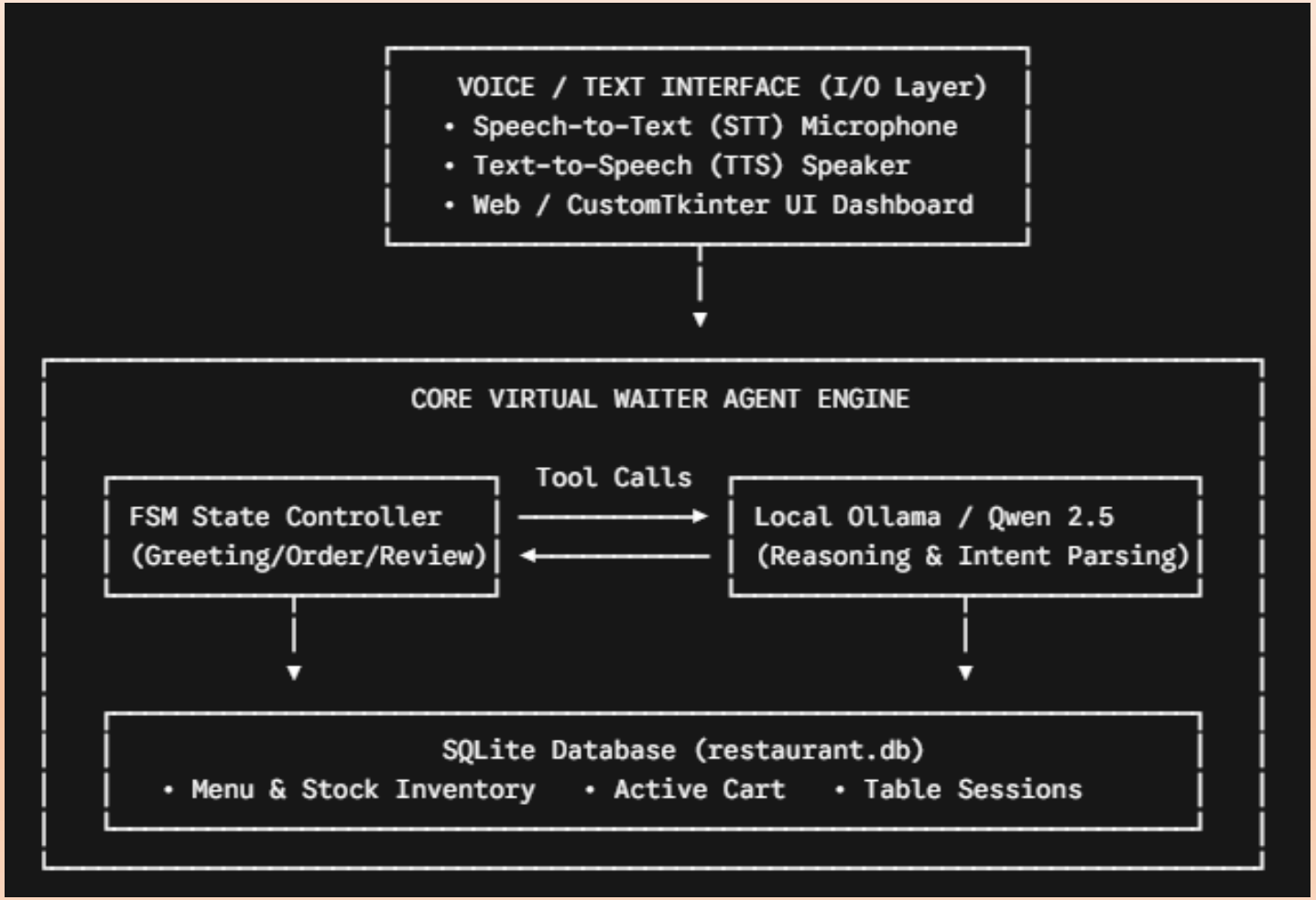
Required File Architecture & Deliverables

Your final code submission must follow this folder structure:

```
ai_virtual_waiter/
│
├── init_restaurant_db.sql     # SQLite schema & seed dataset
├── init_db.py                 # DB initialization script
├── db_tools.py                # Tool functions & SQLite CRUD engine
├── fsm_controller.py          # Finite State Machine state manager
├── speech_engine.py           # Decoupled STT/TTS speech wrappers
├── app.py                     # Main application entry point
├── project_documentation.md   # Architecture, state diagrams & tandem log
│
└── ui_dashboard/              # Web or Desktop GUI files
    ├── templates/             # HTML templates (if using Flask)
    └── static/                # Assets & CSS styling
```

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

System Architecture & Conversation Flow: Detailed Layer-by-Layer Breakdown:



FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

1. The I/O Surface Layer (Top Box)

This is the external boundary of the system that interacts directly with the customer at the table.

- **Speech-to-Text (STT) Microphone:** Listens to physical room audio and transcribes spoken customer requests into plain text strings.
- **Text-to-Speech (TTS) Speaker:** Takes plain string responses generated by the AI agent and synthesizes them into spoken vocal audio for table feedback.
- **Web / CustomTkinter UI Dashboard:** Renders visual elements (food photos, prices, active cart line items, subtotal) and provides manual "human-in-the-loop" touch controls (+, -, Delete).

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

System Architecture & Conversation Flow:

Architectural Role: This layer decouples physical hardware audio drivers and browser graphics from the core agent logic. It converts all incoming customer interactions into a standardized string format before passing them down.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

2. The Core Agent Engine (Middle Box)

This is the internal processing hub where conversation state, artificial intelligence reasoning, and intent execution converge.

- **FSM State Controller (Left Sub-Block):**
  - Acts as the **unbypassable guardrail** enforcing valid business flow (GREETING → ORDERING → REVIEW → CONFIRMED).
  - Prevents state-skipping (e.g., blocking order checkout if the cart is empty or rejecting post-confirmation modifications).
- **Local Ollama / Qwen 2.5 Engine (Right Sub-Block):**
  - Receives natural language text strings from the user alongside the current state context.
  - Evaluates user intent (e.g., *"Add 2 Cheeseburgers with extra pickles"*) and outputs structured JSON function calls matching local database tools.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

2. The Core Agent Engine (Middle Box)

- **Architectural Role:** The bidirectional arrow between FSM Controller and Qwen 2.5 creates a **Reasoning & Tool Execution Loop**. Qwen decides *what action to take*, while the FSM Controller ensures *the action is legal in the current state phase*.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

3. The Relational Persistence Layer (Bottom Database Box)

This serves as the single source of truth for the entire application.

- **Menu & Stock Inventory (menu_items):** Stores item names, descriptions, prices, dietary flags, food photo paths, and real-time inventory counts (stock_qty).
- **Active Cart (cart_items):** Tracks line items, quantities, and special instructions added during the active session.
- **Table Sessions (table_sessions):** Maintains table numbers, creation timestamps, and active state machine stages.
- **Architectural Role:** All tool executions emitted by Qwen directly query or mutate this database via transactional SQLite CRUD functions (db_tools.py). Programmatic inventory guards inside Python check stock_qty before executing commits to guarantee data integrity.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

3. The Relational Persistence Layer (Bottom Database Box)

This serves as the single source of truth for the entire application.

Architectural Role: All tool executions emitted by Qwen directly query or mutate this database via transactional SQLite CRUD functions (db_tools.py). Programmatic inventory guards inside Python check stock_qty before executing commits to guarantee data integrity.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Why This Design Is Robust for each group?

- **Isolation of Failures:** If the microphone stream or TTS voice driver fails, the backend FSM, database transactions, and Qwen tool calling continue running 100% reliably through the text dashboard.
- **Simplified Testing:** Students build and test the entire bottom two layers (FSM + Qwen + SQLite) in **Lesson 6 Lab** via terminal text prompts before connecting the top I/O layer (STT/TTS and UI) for the **Final Project**.

FINAL PROJECT: AI VIRTUAL WAITER FOR PERPETUAL BISTRO

Final Project Specification: AI Virtual Waiter for Perpetual Bistro Using Ollama and Qwen

- Execution Window: October 19 – November 6, 2026
- Milestone 1: Core Engine, Speech Pipeline & FSM Tool Binding

  Due date: October 28, 2026
- Milestone 2: Full System Integration, UI Dashboard & Final Submission

  Due Date: November 6, 2026
- Live Demo Presentations: November 7, 2026